

Análisis de Viabilidad Técnica y Escalabilidad – Aplicación de Ayudas

1. Resumen del Proyecto

La aplicación permitirá gestionar **personas solicitantes de ayuda** y sus **notificaciones**, con posibilidad futura de incorporar categorías, amistades y privacidad. La interfaz incluye:

- Pantalla de inicio de sesión, registro y recuperación de contraseña.
- Pestaña principal con **mapa interactivo**, mostrando notificaciones vinculadas opcionalmente a contenido multimedia y personas.
- Perfil de usuario editable.

Objetivo del análisis: Garantizar que la arquitectura inicial soporte el crecimiento futuro sin requerir rehacer la aplicación.

2. Requerimientos Técnicos Iniciales

Área	Requerimiento
Frontend	App móvil (Flutter o React Native) para iOS y Android; soporte de mapas y contenido multimedia.
Backend	API RESTful o GraphQL; gestión de usuarios, notificaciones y contenido multimedia; seguridad básica.
Base de datos	Inicial: Base de datos relacional (PostgreSQL/MySQL) para personas y notificaciones.
Almacenamiento multimedia	Inicial: AWS S3 / Google Cloud Storage para imágenes y videos.
Autenticación	Email/contraseña, recuperación de contraseña, posible integración futura con OAuth/social login.
Hosting / Infraestructura	Inicial: Servidor en la nube (AWS, GCP o Azure); contenedores Docker opcionales.
Seguridad	HTTPS, cifrado de contraseñas (bcrypt/argon2), validaciones básicas en backend.

3. Viabilidad Técnica

3.1 Frontend

- **Flutter:** Permite una sola base de código para iOS y Android, ideal para MVP.
- **React Native:** Alternativa viable si el equipo domina JS/TS; buena integración con mapas y multimedia.

Riesgos: Actualizaciones de Flutter o React Native pueden romper dependencias si no se mantiene un control de versiones.

3.2 Backend

- **Node.js + Express** o **Spring Boot (Java):** Robustez y escalabilidad; ambos soportan APIs REST y WebSockets (para notificaciones en tiempo real).
- **GraphQL:** Puede ser útil si las notificaciones y usuarios tienen relaciones complejas.

Riesgos: Escalar la lógica de notificaciones puede requerir colas de mensajes (RabbitMQ, Kafka) si la base de usuarios crece mucho.

3.3 Base de Datos

- **PostgreSQL:** Escalable, soporta relaciones complejas y consultas geoespaciales (para el mapa).
- **MySQL:** Más sencillo de configurar; también soporta consultas geoespaciales.

Escalabilidad: Inicialmente suficiente para <100k usuarios; para >1M usuarios considerar particionamiento (sharding) o bases NoSQL para ciertas colecciones (ej: notificaciones multimedia).

3.4 Almacenamiento Multimedia

- Usar **S3 o equivalente** desde el inicio evita sobrecargar la base de datos.
- Versionado y compresión de imágenes para optimizar rendimiento en móvil.

3.5 Infraestructura

- **Docker + Kubernetes:** Ideal si se prevé escalado horizontal.
- **Serverless (AWS Lambda / Firebase Functions):** Alternativa para funciones puntuales (envío de notificaciones, procesamiento de imágenes).

4. Plan de Escalado

Etapa	Objetivo	Tecnología/Medida
MVP (0-10k usuarios)	Funcionalidad básica y estabilidad	Backend monolítico, base de datos relacional, almacenamiento S3.
Crecimiento (10k-100k usuarios)	Soporte a más tráfico	Contenedores Docker, balanceador de carga, caché (Redis/Memcached), colas de mensajes.
Escala avanzada (>100k usuarios)	Escalar globalmente	Microservicios, particionamiento de base de datos, CDN para contenido multimedia, cluster Kubernetes.
Futuro (1M+ usuarios)	Optimización y resiliencia	Servicios serverless para tareas específicas, análisis en tiempo real de notificaciones, replicación multi-región.

5. Recomendaciones

1. **Elegir stack flexible desde el inicio:** Flutter + Node.js/PostgreSQL/S3 permite crecimiento sin rehacer la app.
2. **Diseñar la base de datos pensando en relaciones futuras:** personas, notificaciones, categorías y privacidad.
3. **Implementar API REST clara y modular:** facilita integración con futuras funcionalidades.
4. **Preparar infraestructura escalable:** usar contenedores, almacenamiento externo y cacheo.
5. **Seguridad desde el día 1:** cifrado, autenticación segura, validaciones en backend.